CLOUD-04: AWS Lambda & Serverless Monitoring

> **Series:** CLOUD — Cloud Provider Integrations | **Notebook:** 4 of 8 | **Created:** March 2026 | **Last Updated:** 10/19/2026

Overview

This notebook covers serverless monitoring with Dynatrace, focusing on AWS Lambda. You will learn how to monitor Lambda function performance (cold starts, duration, errors, throttles), integrate API Gateway tracing, analyze Step Functions workflows, assess DynamoDB performance, and build end-to-end serverless application tracing.

Service Detection v2 for Lambda (Early Access in SaaS 1.337; GA per current docs)

Sprint 1.337 SaaS introduced **Service Detection v2 (SDv2) for AWS Lambda** in Early Access; the SDv2 configuration docs now describe it as generally available. It is a major upgrade for serverless monitoring:

1. **Unified rules for OTel and OneAgent.** SDv2 detects Lambda functions whether they emit traces via OneAgent (Lambda layer) or via the OpenTelemetry Lambda extension. The same service entity is enriched from both paths — no more parallel services for the same function.
2. **FaaS-specific metrics.** Sprint 1.337 names them by function only:
   - **Invocation count and failure count**
   - **Invocation duration**
   - **Trigger type breakdown**

They surface in Grail as `dt.service.faas_invoke.count`, split by `faas.trigger` and a boolean `failed` dimension (see the DQL below).
3. **OTel `service.name` enrichment** — when Lambda functions emit OTel spans with `service.name`, Dynatrace uses that value to enrich the SDv2-detected service rather than creating a parallel one.

Sample DQL — Lambda failure rate by trigger type:

```
// dt.faas.failures / dt.faas.invocations / dt.faas.trigger.type do not exist (corrected
// 08/12/2026). The OneAgent FaaS series is dt.service.faas_invoke.count, split by faas.trigger
// with a boolean `failed` dimension — so the failure/total split is a dimension filter, not two
// separate metrics.
timeseries invocations = sum(dt.service.faas_invoke.count), from:-1h, by:{faas.trigger, failed}
| fieldsAdd total = arraySum(invocations)
| summarize {
    failures = sum(if(failed == true, total, else: 0)),
    invocations = sum(total)
  }, by:{faas.trigger}
| fieldsAdd failure_rate_pct = round(failures * 100.0 / invocations, decimals: 2)
| sort failure_rate_pct desc
```

Status: *"SDv2 is generally available for AWS Lambda services monitored by OneAgent. This feature is deactivated by default."* It has a prerequisite: *"SDv2 for AWS Lambda services only works when the [Built-in] FaaS name service detection rule is enabled."* Existing Lambda monitoring via OneAgent layer or OTel extension keeps working.

> **Sources:** SaaS 1.337 (DT docs) — the Early Access introduction and the three metric functions, Configure SDv2 for OneAgent (DT docs) — the GA status and the FaaS-name-rule prerequisite.

Table of Contents

1. Lambda Monitoring Fundamentals
2. Cold Start Detection
3. Lambda Error Analysis
4. Throttle Analysis
5. API Gateway Integration
6. Step Functions Tracing
7. DynamoDB Performance
8. End-to-End Serverless Tracing
9. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`storage:metrics:read`, `storage:entities:read` + `storage:smartscape:read`, `storage:spans:read`, `storage:buckets:read` (Grail IAM permissions)
AWS Integration	AWS cloud integration configured (CLOUD-02)
Lambda Functions	At least one Lambda function with Dynatrace layer or OneAgent extension
Prior Knowledge	CLOUD-01 fundamentals, CLOUD-02 AWS integration

1. Lambda Monitoring Fundamentals

Dynatrace monitors Lambda functions through two complementary approaches:

Approach	Data Source	What It Provides
Cloud Integration	CloudWatch metrics via the Clouds app connection (or a classic ActiveGate on Managed)	Invocations, duration, errors, throttles, concurrent executions
Dynatrace Lambda Layer	OneAgent in Lambda runtime	Distributed traces, code-level visibility, custom metrics

Key Lambda Metrics — the key name depends on how the metrics arrive

Check which ingestion path your environment uses before copying any key from this section. AWS metrics reach Grail by two routes and they use *different key schemes*. A key from the wrong scheme returns an empty result, not an error, so it reads as "this function has no traffic".

Ingestion path	Key scheme	Example
AWS integration (polling)	lowercase, `dt.`-prefixed	`dt.cloud.aws.lambda.invocations`
CloudWatch Metric Streams	CamelCase, no `dt.` prefix, `.By.<Dimension>` suffix	`cloud.aws.lambda.Invocations.By.FunctionName`

Measure	Polling key	Metric Streams key	Healthy range
Invocations	`dt.cloud.aws.lambda.invocations`	`cloud.aws.lambda.Invocations.By.FunctionName`	Application-dependent
Duration (ms)	`dt.cloud.aws.lambda.duration`	`cloud.aws.lambda.Duration.By.FunctionName`	< function timeout
Errors	`dt.cloud.aws.lambda.errors`	`cloud.aws.lambda.Errors.By.FunctionName`	0 (ideally)
Throttles	`dt.cloud.aws.lambda.throttlers`	`cloud.aws.lambda.Throttles.By.FunctionName`	0
Concurrent executions	`dt.cloud.aws.lambda.conc_executions`	`cloud.aws.lambda.ConcurrentExecutions.By.FunctionName`	< reserved concurrency

The DQL cells in this notebook use the Metric Streams scheme, which is what the validation tenant ingests (09/24/2026: 5 distinct `cloud.aws.lambda.*` keys, **0** `dt.cloud.aws.lambda.*`). If your environment polls instead, substitute the left-hand column throughout. Settle it in one query rather than guessing:

```
metrics from:now()-9d
| filter startsWith(metric.key, "cloud.aws.lambda") or startsWith(metric.key, "dt.cloud.aws.lambda")
| summarize n = count(), by:{metric.key}
| sort metric.key asc
```

Group by `metric.key` rather than listing it: bare `metrics` output repeats a key once per series, so a row count is not a key count.

An empty result from *both* prefixes means no Lambda metrics are arriving at all — a different problem from picking the wrong scheme, and worth separating before you debug either.

List Monitored Lambda Functions

> **Sources:** Built-in metrics on Grail (DT docs) — the `builtin:` → `dt.` transformation behind the polling key scheme, CloudWatch Metric Streams (DT docs) — the streaming path that produces the CamelCase `.By.<Dimension>` keys, AWS metrics ingest (DT docs) — the two ingestion routes. Key presence measured on the validation tenant 09/24/2026: 5 distinct Metric Streams keys, 0 polling keys. **Derived:** the side-by-side key mapping is this entry's reconciliation of the two schemes — no page tabulates them together.

```
// List all monitored Lambda functions with runtime and code size
//
// Corrected 08/12/2026: the runtime attribute is awsRuntime, not awsLambdaFunctionRuntime — the
// latter does not exist and failed the whole cell with FIELD_DOES_NOT_EXIST. Time range added
// because dt.entity.* returns only entities seen in the query window, not the standing inventory.
fetch dt.entity.aws_lambda_function, from:-7d
| fieldsKeep id, entity.name, awsRuntime, awsCodeSize, awsMemorySize, awsTimeout, tags
| sort entity.name asc
| limit 25

// Smartscape note (dt.entity.* is deprecated but still functional): AWS Lambda functions ARE
// modeled on Smartscape as smartscapeNodes "AWS_LAMBDA_FUNCTION", but the classic aws* attribute
// fields differ there — inspect smartscapeNodes "AWS_LAMBDA_FUNCTION" | limit 1 for the node
// field names. Keep the classic query above until the fields are mapped.
```

Lambda Execution Time Over Time

```
// Lambda average execution duration over the last 6 hours
//
// Metric keys: these are the CloudWatch Metric Streams keys, named
// cloud.aws.lambda.&lt;CloudWatchName&gt;.By.FunctionName — Invocations / Errors / Duration /
// Throttles / ConcurrentExecutions / IteratorAge. The AWS polling integration produces
// dt.cloud.aws.lambda.* instead (see §1's table); a key from the other scheme returns an empty
// result, not an error. The split dimension is FunctionName — dt.entity.aws_lambda_function is
// null on these metrics. Enumerate what your tenant has, one row per key, with:
//   metrics from:now()-9d | filter startsWith(metric.key, "cloud.aws.lambda") or startsWith(metric.key,
"dt.cloud.aws.lambda") | summarize n = count(), by:{metric.key} | sort metric.key asc
timeseries avgDuration = avg(cloud.aws.lambda.Duration.By.FunctionName), from:-6h, by:{FunctionName}
| fieldsAdd avgDurationValue = arrayAvg(avgDuration)
| sort avgDurationValue desc
| limit 10
```

Tagging this producer — primary Grail tags on Lambda

"A Lambda function doesn't load the ingest enrichment configuration, so its tags come from the DT_TAGS environment variable that you set in the function configuration". Set the keys, space-separated, with their `primary_tags.` prefix:

```
DT_TAGS = "primary_tags.team=checkout primary_tags.app=shop primary_tags.stage=prod"
```

The variable tags the spans, metrics and logs OneAgent emits from the function. Two more things:

- **Account and region are not detected.** *"On serverless platforms, OneAgent can't auto-detect certain primary fields."* Pass `aws.account.id` and `aws.region` in `DT_TAGS` too.
- **The function's Smartscape node is tagged elsewhere.** It comes from the AWS connection, so the CLOUD-02 AWS-tag rule and OpenPipeline promotion tag it — and the `isNull(primary_tags.<key>)` condition there is what keeps that rule from overwriting the `DT_TAGS` value on the function's own telemetry.

> **Sources:** Apply primary Grail tags across an example AWS estate (DT docs) — *"A Lambda function doesn't load the ingest enrichment configuration, so its tags come from the DT_TAGS environment variable that you set in the function configuration"*, Enrich OneAgent telemetry with primary Grail fields and tags (DT docs) — *"On serverless platforms, OneAgent can't auto-detect certain primary fields."*

2. Cold Start Detection

Cold starts occur when Lambda creates a new execution environment. They add latency (100ms to several seconds depending on runtime and package size).

Understanding Cold Starts

Factor	Impact on Cold Start
Runtime	Java/C# > Python/Node.js
Package size	Larger = slower initialization
VPC attachment	Adds ENI creation time (~1-2s)
Provisioned concurrency	Eliminates cold starts (at cost)
SnapStart	Reduces Java cold starts to ~200ms

Detecting Cold Starts via Init Duration

CloudWatch provides an `InitDuration` metric for cold starts. With Dynatrace Lambda Layer, you can detect cold starts from spans.

```
// Detect Lambda cold starts from spans (init phase)
//
// Corrected 08/12/2026. The old cell filtered `contains(span.name, "lambda")`, but a Lambda span
// is named for the FUNCTION, not the word "lambda" — so the filter discarded every matching span
// and the cell reported cold_start_count = 0 while 14 real cold starts (avg 2.45 s) sat in the
// window. A zero inside a summarize row is the easiest kind of wrong answer to miss: the cell
// returns a row, it just returns a false one. faas.coldstart alone is the correct discriminator.
// Also dropped faas.invocation_id, which is not a field (faas.* has no invocation_id).
fetch spans, from:-6h
| filter faas.coldstart == true
| summarize {cold_start_count = count(), avg_duration_ms = avg(duration) / 1ms}, by:{faas.name}
| sort cold_start_count desc
| limit 20
```

Lambda Invocations with Duration Percentiles

```
// Lambda duration percentiles over the last 6 hours — computed from spans, not the metric.
//
// Corrected 08/12/2026 (two defects). The old cell called
// percentile(dt.cloud.aws.lambda.duration, 50): the key does not exist, AND percentile() does not
// work on the real CloudWatch Duration metric either — cloud.aws.lambda.Duration.By.FunctionName
// is a pre-aggregated gauge, so percentile() over it returns an empty result even when the key is
// spelled correctly. Only avg/min/max/sum are meaningful there. Real percentiles need the
// underlying distribution, which lives on the spans.
//
// Corrected 10/02/2026: filter to the invocation span (span.kind == "server"). faas.name is also
// carried by the function's outbound and internal child spans, so counting every span with
// faas.name over-counted invocations ~8x on the validation tenant (2,872 spans vs 359 server
// spans in 6 h, against 351 CloudWatch invocations) and mixed child-span durations into the
// percentiles.
fetch spans, from:-6h
| filter isNotNull(faas.name) and span.kind == "server"
| summarize {
    p50 = percentile(duration, 50) / 1ms,
    p90 = percentile(duration, 90) / 1ms,
    p99 = percentile(duration, 99) / 1ms,
    invocations = count()
  }, by:{faas.name}
| sort p99 desc
| limit 10
```

3. Lambda Error Analysis

Lambda errors fall into two categories:

Error Type	Description	Metric
Function errors	Unhandled exceptions, timeout, OOM	`dt.cloud.aws.lambda.errors`
Invocation errors	Permission issues, throttling, invalid payload	`dt.cloud.aws.lambda.throttlers`

> **Breaking (SaaS 1.346 — staged rollout from 08/25/2026): expect your Lambda failure rate to rise.** Verbatim: *"HTTP failure detection now applies to FaaS services (AWS Lambda, Azure Functions, and GCP Cloud Functions) with HTTP data. After this update, you may see a higher failure rate in your environment. This is expected—issues with these services are now detected and reported for the first time."*
>
> The rise is **new detection, not new breakage** — HTTP-level failures your functions were already returning are now reported as failures. Three practical consequences:
>
> - **Re-baseline before you alert.** Any static failure-rate threshold, SLO, or anomaly baseline covering FaaS services was tuned against the pre-1.346 numbers. Let the new rate settle before treating a step change as an incident — and expect Davis baselines to re-learn.

> - **Do not chase the step change as a regression.** The first post-rollout spike is the most likely moment for a false escalation. Confirm the tenant version before opening an investigation into a failure-rate jump.
> - **The step is a one-time level shift, not a trend.** Compare like-for-like windows on the same side of the rollout.
>
> This applies to the metric-based error analysis below and to trace-based failure analysis alike, and it is a cross-cloud change — Azure Functions and GCP Cloud Functions are named alongside Lambda, so the same re-baselining applies in CLOUD-05 and CLOUD-06.
>
> **Sources:** SaaS 1.346 (DT docs).

Error Rate by Function

```
// Lambda error count by function over the last 24 hours
//
// Metric keys: these are the CloudWatch Metric Streams keys, named
// cloud.aws.lambda.&lt;CloudWatchName&gt;.By.FunctionName — Invocations / Errors / Duration /
// Throttles / ConcurrentExecutions / IteratorAge. The AWS polling integration produces
// dt.cloud.aws.lambda.* instead (see §1's table); a key from the other scheme returns an empty
// result, not an error. The split dimension is FunctionName — dt.entity.aws_lambda_function is
// null on these metrics. Enumerate what your tenant has, one row per key, with:
//   metrics from:now()-9d | filter startsWith(metric.key, "cloud.aws.lambda") or startsWith(metric.key,
"dt.cloud.aws.lambda") | summarize n = count(), by:{metric.key} | sort metric.key asc
timeseries errors = sum(cloud.aws.lambda.Errors.By.FunctionName), from:-24h, by:{FunctionName}
| fieldsAdd totalErrors = arraySum(errors)
| filter totalErrors &gt; 0
| sort totalErrors desc
| limit 10
```

Error-to-Invocation Ratio

```
// Error rate percentage by function over the last 24 hours
//
// Corrected 08/12/2026 (two defects). Besides the non-existent metric keys, the old cell used
// `append [ ... ]` to bring invocations alongside errors. append STACKS ROWS — it does not join
// columns — so totalErrors and totalInvocations never landed on the same row and error_pct was
// null for every row even with valid keys. Request both series inside ONE timeseries block, which
// is what actually aligns them on a shared timeline and grouping.
//
// Metric keys: these are the CloudWatch Metric Streams keys, named
// cloud.aws.lambda.&lt;CloudWatchName&gt;.By.FunctionName — Invocations / Errors / Duration /
// Throttles / ConcurrentExecutions / IteratorAge. The AWS polling integration produces
// dt.cloud.aws.lambda.* instead (see §1's table); a key from the other scheme returns an empty
// result, not an error. The split dimension is FunctionName — dt.entity.aws_lambda_function is
// null on these metrics. Enumerate what your tenant has, one row per key, with:
//   metrics from:now()-9d | filter startsWith(metric.key, "cloud.aws.lambda") or startsWith(metric.key,
"dt.cloud.aws.lambda") | summarize n = count(), by:{metric.key} | sort metric.key asc
timeseries {
  invocations = sum(cloud.aws.lambda.Invocations.By.FunctionName),
  errors = sum(cloud.aws.lambda.Errors.By.FunctionName)
}, from:-24h, by:{FunctionName}
| fieldsAdd totalInvocations = arraySum(invocations), totalErrors = arraySum(errors)
| filter totalInvocations &gt; 0
| fieldsAdd error_pct = round((totalErrors / totalInvocations) * 100, decimals: 2)
| sort error_pct desc
| limit 10
```

4. Throttle Analysis

Throttling occurs when Lambda cannot allocate an execution environment, typically due to:
- **Account-level concurrency limit** (default 1,000 per region)
- **Reserved concurrency** on the function being exhausted
- **Burst limit** exceeded (3,000 in US regions, varies by region)

Throttled Invocations Over Time

```
// Lambda throttles over the last 24 hours by function
//
// The `filter totalThrottles &gt; 0` guard was dropped (08/12/2026) — on a healthy estate every function
// throttles zero times, so the guard emptied the table and made a correct result look like a
// broken query. Seeing the zeros is the point.
//
// Metric keys: these are the CloudWatch Metric Streams keys, named
// cloud.aws.lambda.&lt;CloudWatchName&gt;.By.FunctionName — Invocations / Errors / Duration /
// Throttles / ConcurrentExecutions / IteratorAge. The AWS polling integration produces
// dt.cloud.aws.lambda.* instead (see §1's table); a key from the other scheme returns an empty
// result, not an error. The split dimension is FunctionName — dt.entity.aws_lambda_function is
// null on these metrics. Enumerate what your tenant has, one row per key, with:
//   metrics from:now()-9d | filter startsWith(metric.key, "cloud.aws.lambda") or startsWith(metric.key,
"dt.cloud.aws.lambda") | summarize n = count(), by:{metric.key} | sort metric.key asc
timeseries throttles = sum(cloud.aws.lambda.Throttles.By.FunctionName), from:-24h, by:{FunctionName}
| fieldsAdd totalThrottles = arraySum(throttles)
| fields FunctionName, totalThrottles
| sort totalThrottles desc
| limit 20
```

Concurrent Executions Trend

```
// Concurrent Lambda executions over the last 6 hours
//
// Metric keys: these are the CloudWatch Metric Streams keys, named
// cloud.aws.lambda.&lt;CloudWatchName&gt;.By.FunctionName — Invocations / Errors / Duration /
// Throttles / ConcurrentExecutions / IteratorAge. The AWS polling integration produces
// dt.cloud.aws.lambda.* instead (see §1's table); a key from the other scheme returns an empty
// result, not an error. The split dimension is FunctionName — dt.entity.aws_lambda_function is
// null on these metrics. Enumerate what your tenant has, one row per key, with:
//   metrics from:now()-9d | filter startsWith(metric.key, "cloud.aws.lambda") or startsWith(metric.key,
"dt.cloud.aws.lambda") | summarize n = count(), by:{metric.key} | sort metric.key asc
timeseries concurrency = max(cloud.aws.lambda.ConcurrentExecutions.By.FunctionName), from:-6h, by:{FunctionName}
| fieldsAdd peakConcurrency = arrayMax(concurrency)
| sort peakConcurrency desc
| limit 10
```

5. API Gateway Integration

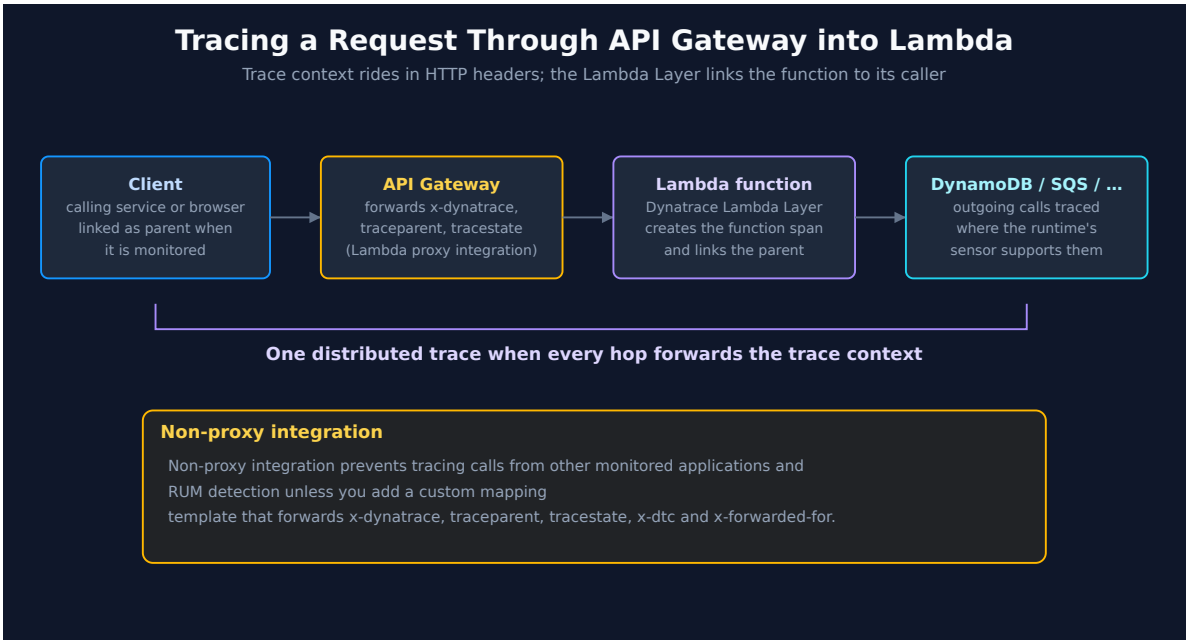
AWS API Gateway is a common front-end for Lambda functions. Dynatrace monitors the API Gateway-to-Lambda path for end-to-end latency.

API Gateway Monitoring Points

Metric	What It Measures
Count	Total API requests
Latency	End-to-end latency (API GW → Lambda → response)
IntegrationLatency	Lambda execution time only
4XXError	Client-side errors
5XXError	Server-side errors (Lambda failures)

Tracing API Gateway Requests

With the Dynatrace Lambda Layer, traces propagate through API Gateway into Lambda:



> **Sources:** Trace Lambda functions (DT docs).

```
// Trace spans from API Gateway and Lambda in the last hour
//
// Corrected 08/12/2026: the old `contains(span.name, "lambda") or contains(span.name,
// "api-gateway")` name-guessing filter matched nothing, because spans are named for the function
// or route. Select on the semantic field instead — faas.name is present on every FaaS span.
fetch spans, from:-1h
| filter span.kind == "server" and isNotNull(faas.name)
| fieldsKeep start_time, trace.id, span.name, duration, span.status_code, faas.name
| sort start_time desc
| limit 20
```

6. Step Functions Tracing

AWS Step Functions orchestrate Lambda functions into workflows. Dynatrace can trace the entire state machine execution.

Step Functions Monitoring

What to Monitor	Why
Execution duration	Detect slow workflows
Failed executions	Identify error patterns
State transitions	Find bottleneck states
Lambda cold starts within workflows	Impact on workflow latency

Step Function Execution Tracking via Logs

```
// Step Function execution logs (if forwarded to Dynatrace)
fetch logs, from:-24h
| filter contains(content, "StepFunction") or contains(content, "StateMachine")
| fieldsKeep timestamp, content, log.source
| sort timestamp desc
| limit 20
```

7. DynamoDB Performance

DynamoDB is frequently used with Lambda for serverless data storage. Key monitoring areas:

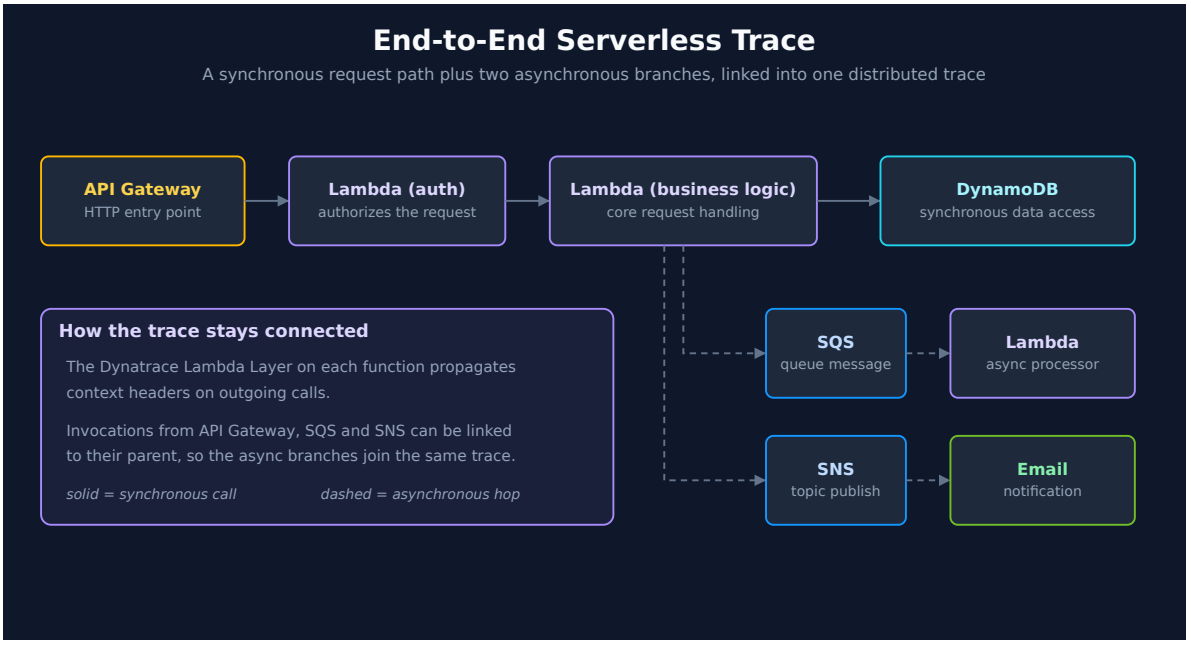
Metric	Concern
Read/Write capacity	Throttling if exceeded
Latency	Slow queries impacting Lambda duration
Consumed vs provisioned	Over/under-provisioning
Error count	Conditional check failures, throttles

DynamoDB Span Analysis

```
// Database call duration from Lambda spans in the last 6 hours
//
// Corrected 08/12/2026: `db.operation` is not a field — the stable name is `db.operation.name`.
// It went unnoticed because the preceding `db.system == "dynamodb"` filter already emptied the
// stream, so the bad field in the summarize was never evaluated. The hard-coded dynamodb filter
// is also removed: it returns nothing unless that engine is present. Group by db.system and read
// what your estate actually calls.
fetch spans, from:-6h
| filter span.kind == "client" and isNotNull(db.system)
| summarize {avg_duration_ms = avg(duration) / 1ms, call_count = count()}, by:{db.system, db.namespace,
db.operation.name}
| sort avg_duration_ms desc
| limit 15
```

8. End-to-End Serverless Tracing

A complete serverless application might follow this pattern:



Dynatrace traces the entire flow as a single distributed trace when the Lambda Layer propagates context headers.

Trace Analysis Across Services

```
// Slowest traces that include a Lambda invocation, last hour.
//
// Corrected 09/24/2026: the old cell filtered only span.kind == "server" and never selected on
// Lambda, so it ranked every single-span server trace in the estate. faas.name marks the Lambda
// spans; the join then measures each of those traces end to end.
fetch spans, from:-1h
| filter isNotNull(faas.name)
| fields trace.id
| join [
    fetch spans, from:-1h
    | summarize {trace_duration = max(duration), span_count = count()}, by:{trace.id}
  ], on:{trace.id}, fields:{trace_duration, span_count}
| dedup trace.id
| sort trace_duration desc
| limit 10
```

Serverless Monitoring Best Practices

Practice	Description
Always install Dynatrace Lambda Layer	Enables distributed tracing and code-level visibility
Set meaningful function names	Avoid auto-generated names for better identification
Monitor concurrency headroom	Alert before hitting account limits
Track cold start ratio	High ratio indicates scaling or provisioned concurrency needs
Correlate API Gateway with Lambda	End-to-end latency matters more than Lambda duration alone

9. Summary and Next Steps

Key Takeaways

- Lambda monitoring combines **CloudWatch metrics** (invocations, errors, throttles) with **Dynatrace Lambda Layer** (traces, code-level visibility)
- **Cold starts** can be detected via spans and mitigated with provisioned concurrency or SnapStart
- **Error-to-invocation ratio** is a more meaningful health metric than raw error count
- **End-to-end tracing** through API Gateway, Lambda, and downstream services requires context propagation

Next Steps

- **CLOUD-05: Azure Integration** — Azure-specific monitoring patterns
- **CLOUD-07: CloudWatch Log Ingestion** — Forward Lambda logs to Dynatrace
- **CLOUD-08: Multi-Cloud Patterns** — Unified serverless monitoring across providers

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.